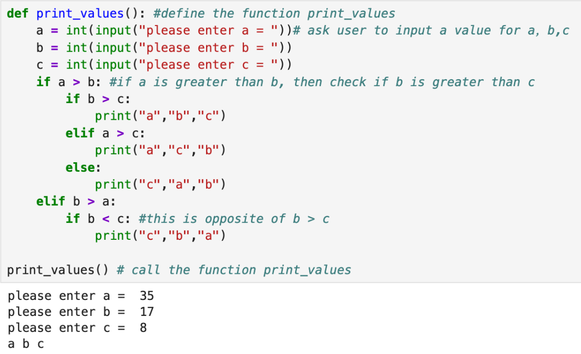
ESE5023 Assignment 01 Report

Ruochen, Liu 12631508

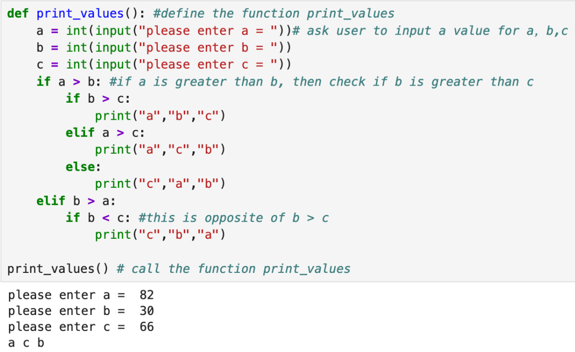
- ## PS1_1

I enter several values of a, b, c to print different outputs, as shown in the following figures:



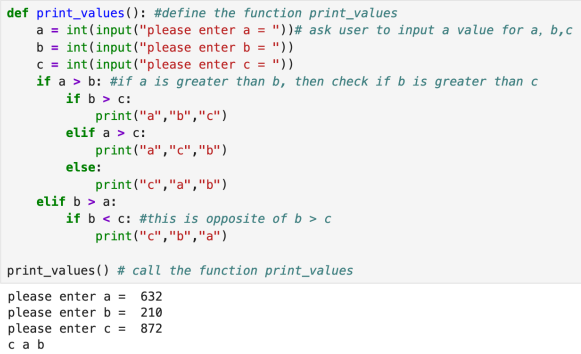
```python
def print_values(): #define the function print_values
    a = int(input("please enter a = "))# ask user to input a value for a, b,c
    b = int(input("please enter b = "))
    c = int(input("please enter c = "))
    if a > b: #if a is greater than b, then check if b is greater than c
        if b > c:
            print("a","b","c")
        elif a > c:
            print("a","c","b")
        else:
            print("c","a","b")
    elif b > a:
        if b < c: #this is opposite of b > c
            print("c","b","a")

print_values() # call the function print_values
```

```
please enter a =  35
please enter b =  17
please enter c =  8
a b c
```



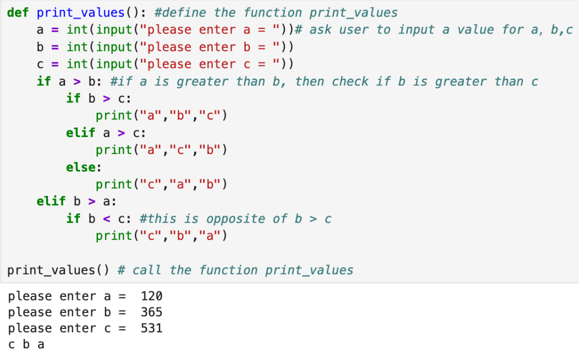
```python
def print_values(): #define the function print_values
    a = int(input("please enter a = "))# ask user to input a value for a, b,c
    b = int(input("please enter b = "))
    c = int(input("please enter c = "))
    if a > b: #if a is greater than b, then check if b is greater than c
        if b > c:
            print("a","b","c")
        elif a > c:
            print("a","c","b")
        else:
            print("c","a","b")
    elif b > a:
        if b < c: #this is opposite of b > c
            print("c","b","a")

print_values() # call the function print_values
```

```
please enter a =  82
please enter b =  30
please enter c =  66
a c b
```



```python
def print_values(): #define the function print_values
    a = int(input("please enter a = "))# ask user to input a value for a, b,c
    b = int(input("please enter b = "))
    c = int(input("please enter c = "))
    if a > b: #if a is greater than b, then check if b is greater than c
        if b > c:
            print("a","b","c")
        elif a > c:
            print("a","c","b")
        else:
            print("c","a","b")
    elif b > a:
        if b < c: #this is opposite of b > c
            print("c","b","a")

print_values() # call the function print_values
```

```
please enter a =  632
please enter b =  210
please enter c =  872
c a b
```

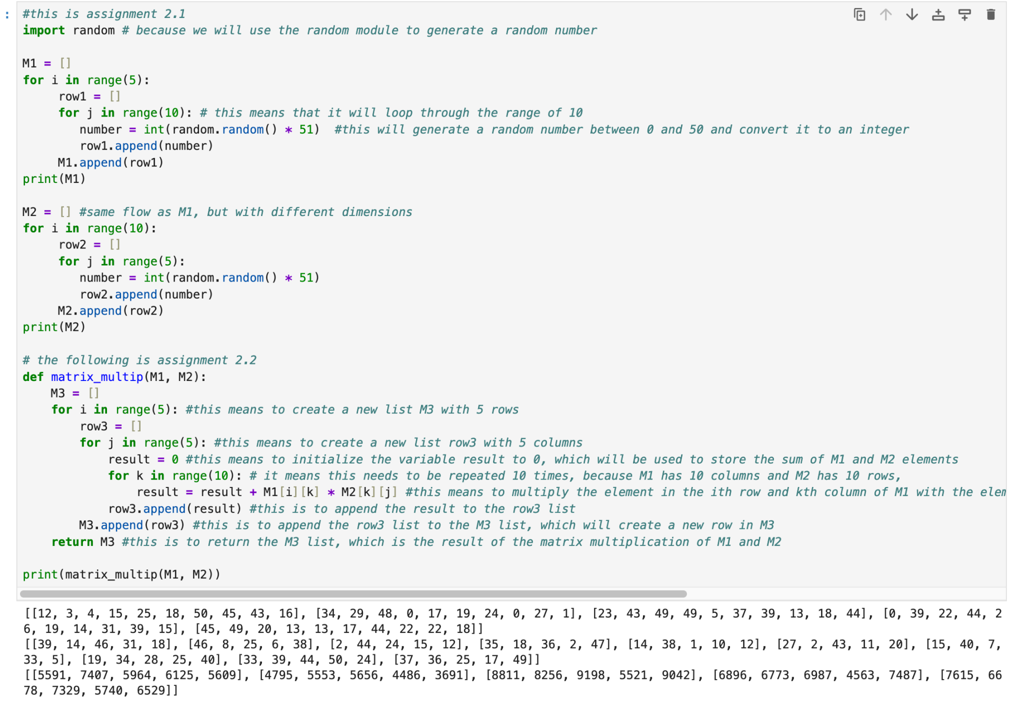


```python
def print_values(): #define the function print_values
    a = int(input("please enter a = "))# ask user to input a value for a, b,c
    b = int(input("please enter b = "))
    c = int(input("please enter c = "))
    if a > b: #if a is greater than b, then check if b is greater than c
        if b > c:
            print("a","b","c")
        elif a > c:
            print("a","c","b")
        else:
            print("c","a","b")
    elif b > a:
        if b < c: #this is opposite of b > c
            print("c","b","a")

print_values() # call the function print_values
```

```
please enter a =  120
please enter b =  365
please enter c =  531
c b a
```

Some notes are commented near the code.

- ## PS1_2



```python
#this is assignment 2.1
import random # because we will use the random module to generate a random number

M1 = []
for i in range(5):
     row1 = []
     for j in range(10): # this means that it will loop through the range of 10
        number = int(random.random() * 51)  #this will generate a random number between 0 and 50 and convert it to an integer
        row1.append(number)
     M1.append(row1)
print(M1)

M2 = [] #same flow as M1, but with different dimensions
for i in range(10):
     row2 = []
     for j in range(5):
        number = int(random.random() * 51)
        row2.append(number)
     M2.append(row2)
print(M2)

# the following is assignment 2.2
def matrix_multip(M1, M2):
    M3 = []
    for i in range(5): #this means to create a new list M3 with 5 rows
        row3 = []
        for j in range(5): #this means to create a new list row3 with 5 columns
            result = 0 #this means to initialize the variable result to 0, which will be used to store the sum of M1 and M2 elements
            for k in range(10): # it means this needs to be repeated 10 times, because M1 has 10 columns and M2 has 10 rows,
                result = result + M1[i][k] * M2[k][j] #this means to multiply the element in the ith row and kth column of M1 with the elem
            row3.append(result) #this is to append the result to the row3 list
        M3.append(row3) #this is to append the row3 list to the M3 list, which will create a new row in M3
    return M3 #this is to return the M3 list, which is the result of the matrix multiplication of M1 and M2

print(matrix_multip(M1, M2))
```

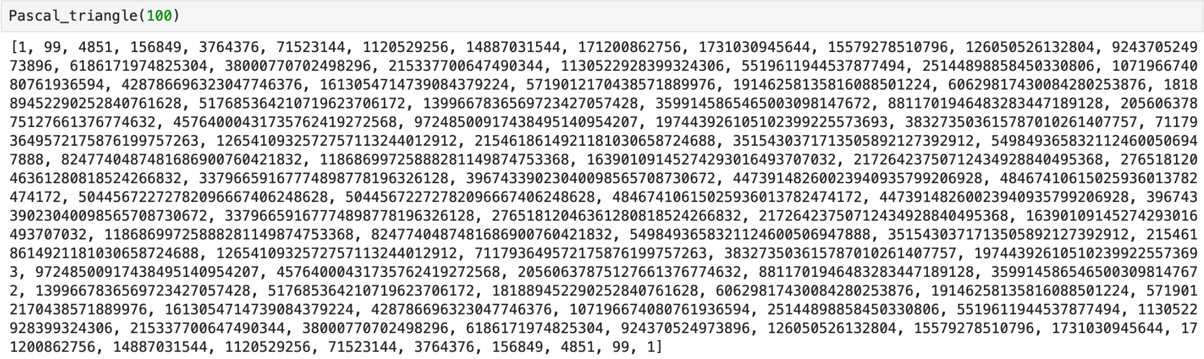
```
[[12, 3, 4, 15, 25, 18, 50, 45, 43, 16], [34, 29, 48, 0, 17, 19, 24, 0, 27, 1], [23, 43, 49, 49, 5, 37, 39, 13, 18, 44], [0, 39, 22, 44, 2
6, 19, 14, 31, 39, 15], [45, 49, 20, 13, 13, 17, 44, 22, 22, 18]]
[[39, 14, 46, 31, 18], [46, 8, 25, 6, 38], [2, 44, 24, 15, 12], [35, 18, 36, 2, 47], [14, 38, 1, 10, 12], [27, 2, 43, 11, 20], [15, 40, 7,
33, 5], [19, 34, 28, 25, 40], [33, 39, 44, 50, 24], [37, 36, 25, 17, 49]]
[[5591, 7407, 5964, 6125, 5609], [4795, 5553, 5656, 4486, 3691], [8811, 8256, 9198, 5521, 9042], [6896, 6773, 6987, 4563, 7487], [7615, 66
78, 7329, 5740, 6529]]
```

Here I reviewed the rule of matrix multiplication by online source:

https://mp.weixin.qq.com/s?__biz=MzI3MzE3OTI0Mw==&mid=2247581408&idx=1&sn=478f9b77e92750abb7794498aa228eac&chksm=ea1f4e85739f66a2e4be78925da1d0ebfc8486769e93827319057484f3d80447a1d2c867fb6a&scene=27

- **PS1_3**

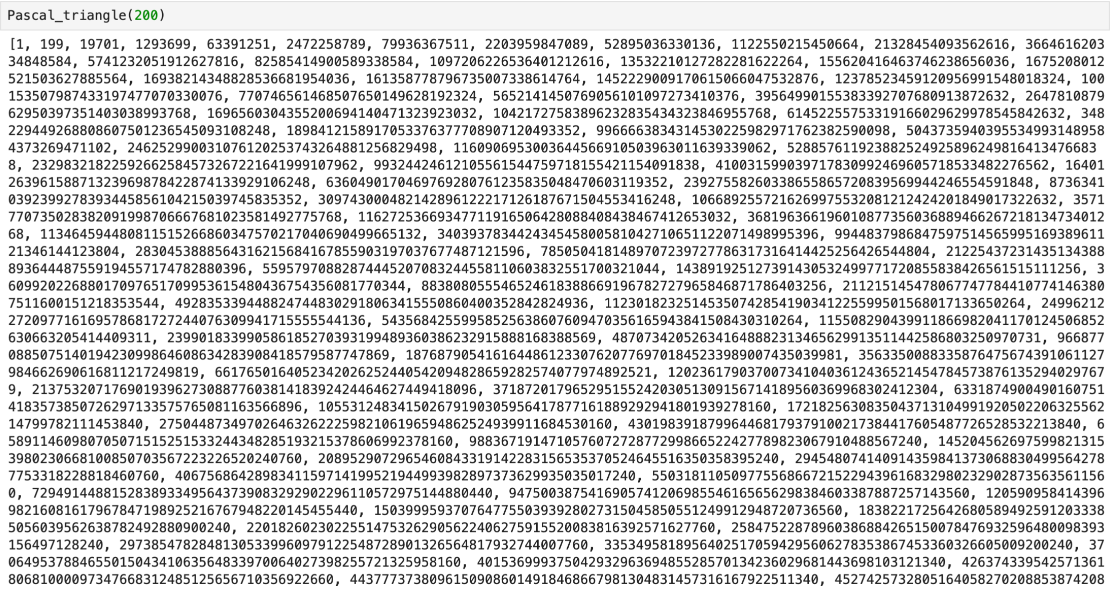
This this Pascal triangle 100:



```
Pascal_triangle(100)
```

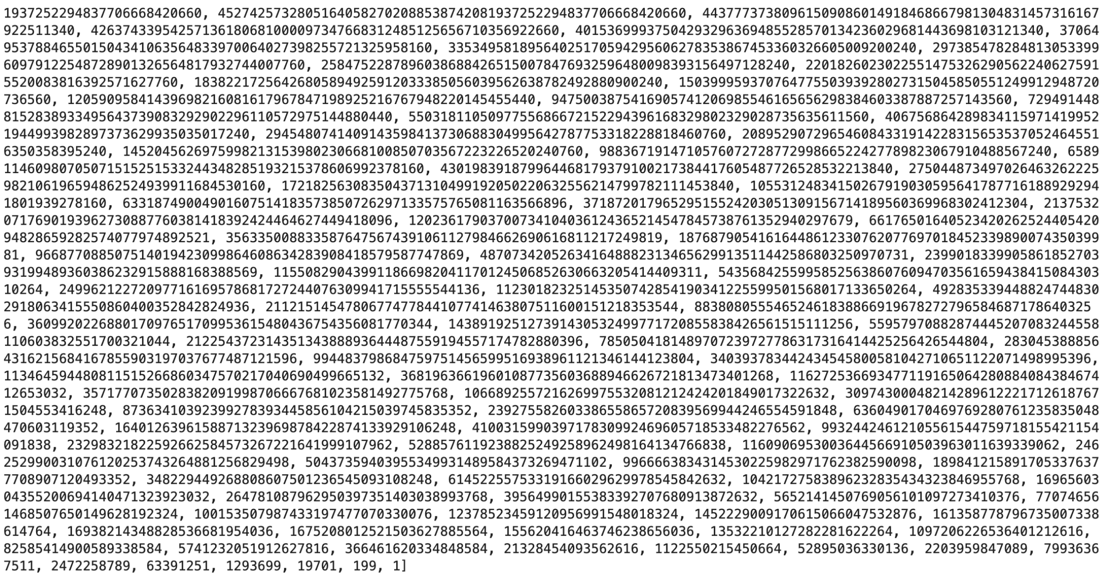
```
[1, 99, 4851, 156849, 3764376, 71523144, 1120529256, 14887031544, 171200862756, 1731030945644, 15579278510796, 126050526132804, 9243705249
73896, 6186171974825304, 38000770702498296, 215337700647490344, 1130522928399324306, 5519611944537877494, 25144898858450330806, 1071966740
80761936594, 428786696323047746376, 1613054714739084379224, 5719012170438571889976, 19146258135816088501224, 60629817430084280253876, 1818
89452290252840761628, 517685364210719623706172, 1399667836569723427057428, 3599145865465003098147672, 8811701946483283447189128, 205606378
75127661376774632, 457640004317357624192725568, 972485009174384951409542207, 1974439261051023992255736930, 3832735036157870102614077570, 71179
3649572175876199757263, 12654109325727571132440129120, 21546186149211810306587246880, 35154303717135058921273929120, 549849365832112460050694
7888, 82477404874816869007604218320, 118686997258882811498747533680, 163901091452742930164937070320, 217264237507124349288404953680, 276518120
4636128081852426683200, 337966591677748987781963261280, 396743390230400985657087306720, 447391482600239409357992069280, 484674106150259360137820
474172, 504456722727820966674062486280, 504456722727820966674062486280, 484674106150259360137820474172, 447391482600239409357992069280, 396743
390230400985657087306720, 337966591677748987781963261280, 276518120463612808185242668320, 217264237507124349288404953680, 163901091452742930160
493707032, 118686997258882811498747533680, 82477404874816869007604218320, 54984936583211246005069478880, 35154303717135058921273929120, 215461
86149211810306587246880, 12654109325727571132440129120, 7117936495721758761997572630, 3832735036157870102614077570, 197443926105102399225573690
3, 972485009174384951409542070, 457640004317357624192725680, 205606378751276613767746320, 88117019464832834471891280, 359914586546500309814767
2, 1399667836569723427057428, 517685364210719623706172, 181889452290252840761628, 60629817430084280253876, 19146258135816088501224, 571901
2170438571889976, 1613054714739084379224, 428786696323047746376, 107196674080761936594, 25144898858450330806, 5519611944537877494, 1130522
928399324306, 215337700647490344, 38000770702498296, 6186171974825304, 924370524973896, 126050526132804, 15579278510796, 1731030945644, 17
1200862756, 14887031544, 1120529256, 71523144, 3764376, 156849, 4851, 99, 1]
```

This this Pascal triangle 200:



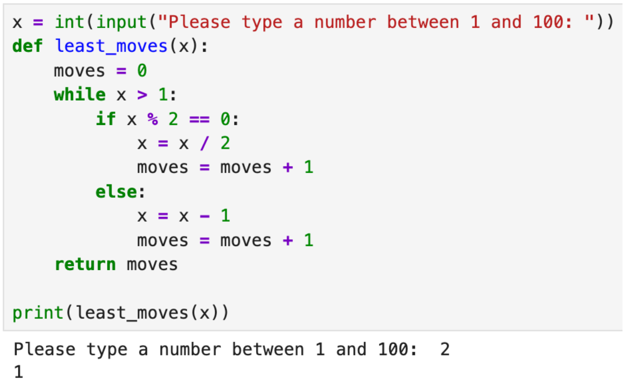
```
Pascal_triangle(200)
```

```
[1, 199, 19701, 1293699, 63391251, 2472258789, 79936367511, 2203959847089, 52895036330136, 1122550215450664, 21328454093562616, 3664616203
34848584, 5741232051912627816, 82585414900589338584, 1097206226536401212616, 13532210127282281622264, 155620416463746238656036, 1675208012
521503627885564, 16938214348828536681954036, 161358778796735007338614764, 1452229009170615066047532876, 12378523459120956991548018324, 100
1535079874331974770703300760, 7707465614685076501496281923240, 56521414507690561010972734103760, 395649901553833927076809138726320, 2647810879
629503973514030389937680, 16965603043552006941404713239230320, 104217275838962328354343238469557680, 614522557533191660296299785458426320, 348
22944926880860750123654509310824800, 189841215891705337637770890712049335200, 996666383431453022598297176238259009800, 504373594039553499314895800
4373269471102, 24625299003107612025374326488125682949800, 11609069530036445669105039630116393390620, 528857611923882524925896249816413476683
8, 232983218225926625845732672216419991079620, 993244246121055615447597181554211540918380, 410031599039717830992469605718533482276562, 16401
26396158871323969878422874133929106248, 6360490170469769280761235835048470603119352, 2392755826033865586572083956994424655459184800, 8736341
03923992783934458561042150397458353520, 3097430004821428961222171261876715045534162480, 10668925572162699755320812124242018490173226320, 3571
77073502838209199870666768102358149277576800, 1162725366934771191650642808840843846741265303200, 3681963661960108773560368894662672181347340120
68, 11346459448081151526686034757021704069049966513200, 34039378344243454580058104271065112207149899539600, 99448379868475975145659951693896110
21346144123804, 283045388856431621568416785590319703767748712159600, 785050418148970723972778631731641442525642654480400, 2122543723143513438800
893644487559194557174782880396, 559579708828744452070832445581106038325517003210440, 1438919251273914305324997717208558384265615151112560, 3
60992022688017097651709953615480436754356081770344, 883808055546524618388669196782727965846871786403256, 211215145478067747784410774146380
7511600151218353544, 4928353394488247448302918063415550860400352842824936, 11230182325145350742854190341225599501568017133650264, 24996212
2720977161695786817272440763099417155555441360, 54356842559958525638607609470356165943841508430310264, 115508290439911866982041170124506852
6306632054144093110, 239901833990586185270393199489360386232915888168388569, 48707342052634164888231346562991351144258680325097073100, 966877
0885075140194230998646086342839084185795877478690, 187687905416164486123307620776970184523398900743503998100, 356335008833587647567439106112700
98466269061681121724981900, 661765016405234202625244054209482865928257407797489252100, 120236179037007341040361243652145478457387613529402976700
9, 21375320717690193962730887760381418392424464627449418096, 3718720179652951552420305130915671418956036996830241230400, 633187490049016075100
4183573850726297133575765081163566896, 1055312483415026791903059564178771618892929418019392781600, 172182563083504371310499192050220632556200
14799782111453840, 275044873497026463262225982106196594862524939911684530160, 430198391879964468179379100217384417605487726528532213840, 6
58911460980705071515251533244348285193215378606992378160, 988367191471057607272877299866522427789823067910488567240, 145204562697599821315
398023066810085070356722322652024076000, 208952907296546084331914228315653537052464551635035839524000, 294548074140914359841373068830499564278
775331822881846076000, 406756864289834115971419952194499398289737362993503501724000, 550318110509775568667215229439616832980232902873563561156
0, 729491448815283893349564373908329290229611057297514488044000, 947500387541690574120698554616565629838460338788725714356000, 120590958414396
98216081617967847198925216767948220145455440, 150399959370764775503939280273150458505512499129487207365600, 1838221725642680589492591203338
5056039562638782492880900240, 22018260230225514753262905622406275915520083816392571627760, 258475228789603868842651500784769325964800983930
156497128240, 29738547828481305339960979122548728901326564817932744007760, 33534958189564025170594295606278353867453360326605009200240, 37
0649537884655015043410635648339700640273982557213259581600, 401536999375042932963694855285701342360296814436981031213400, 426374339542571361
80681000097347668312485125656710356922660, 443777373809615090860149184686679813048314573161679225113400, 4527425732805164058270208853874208
```



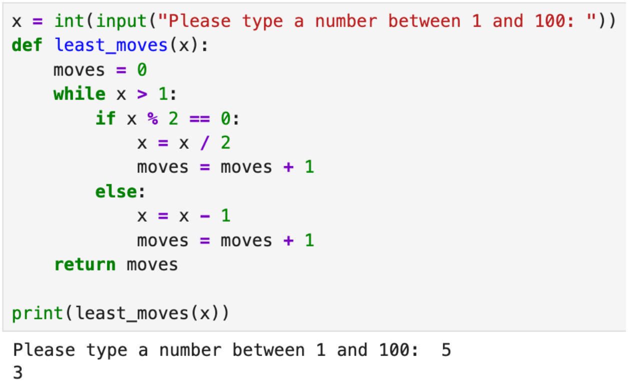
```
19372522948377066684206600, 452742573280516405827020885387420819372522948377066684206600, 443777373809615090860149184686679813048314573161670
922511340, 426374339542571361806810000973476683124851256567103569226600, 401536999375042932963694855285701342360296814436981031213400, 37064
95378846550150434106356483397006402739825572132595816000, 335349581895640251705942956062783538674533603266050092002400, 297385478284813053399
60979122548728901326564817932744007760, 258475228789603868842651500784769325964800983931564971282400, 220182602302255147532629056224062759100
55200838163925716277600, 183822172564268058949259120333850560395626387824928809002400, 150399959370764775503939280273150458505512499129487200
736560, 120590958414396982160816179678471989252167679482201454554400, 947500387541690574120698554616565629838460338788725714356000, 729491448
8152838933495643739083292902296110572975144880440, 5503181105097755686672152294396168329802329028735635611560, 406756864289834115971419952
194499398289737362993503501724000, 294548074140914359841373068830499564278775331822881846076000, 208952907296546084331914228315653537052464551
6350358395240, 14520456269759982131539802306681008507035672232265202407600, 988367191471057607272877299866522427789823067910488567240, 6589
114609807050715152515332443482851932153786069923781600, 430198391879964468179379100217384417605487726528532213840, 275044873497026463262225
982106196594862524939911684530160, 172182563083504371310499192050220632556214799782111453840, 105531248341502679190305956417877161889292940
1801939278160, 633187490049016075141835738507262971335757650811635668960, 371872017965295155242030513091567141895603699683024123040, 2137532
07176901939627308877603814183924244646274494180960, 120236179037007341040361243652145478457387613529402976790, 661765016405234202625244054200
948286592825740779748925210, 35633500883358764756743910611279846626906168112172498190, 187687905416164486123307620776970184523398900743503990
81, 96687708850751401942309986460863428390841857958774786900, 48707342052634164888231346562991351144258680325097073100, 239901833990586185270
39319948936038623291588816838856900, 115508290439911866982041170124506852630663205414409311, 543568425599585256386076094703561659438415084303
10264, 2499621227209771616957868172724407630994171555554413600, 11230182325145350742854190341225599501568017133650264, 492835339448824744830
29180634155508604003528428249360, 211215145478067747784410774146380751160015121835354400, 88380805554652461838866919678272796584687178640325
6, 36099202268801709765170995361548043675435608177034400, 1438919251273914305324997717208558384265615151112560, 559579708828744452070832445580
11060383255170032104400, 21225437231435134388893644487559194557174782880396, 785050418148970723972778631731641442525642654480400, 28304538885600
43162156841678559031970376774871215960, 99448379868475975145659951693896112134614412380400, 34039378344243454580058104271065112207149899539600,
11346459448081151526686034757021704069049966513200, 368196366196010877356036889466267218134734012680, 116272536693477119165064280884084384674
12653032, 35717707350283820919987066676810235814927757680, 10668925572162699755320812124242018490173226320, 30974300048214289612221712618767
1504553416248, 87363410392399278393445856104215039745835352, 2392755826033865586572083956994424655459184800, 636049017046976928076123583504800
470603119352, 16401263961588713239698784228741339291062480, 41003159903971783099246960571853348227656200, 993244246121055615447597181554211540
091838, 2329832182259266258457326722164199910796200, 528857611923882524925896249816413476683800, 1160906953003644566910503963011639339062, 246
2529900310761202537432648812568294980, 50437359403955349931489584373269471102, 99666638343145302259829717623825900980, 1898412158917053376370
77089071204933520, 3482294492688086075012365450931082480, 614522557533191660296299785458426320, 1042172758389623283543432384695576800, 16965603
04355200694140471323923032, 2647810879629503973514030389937680, 395649901553833927076809138726320, 56521414507690561010972734103760, 77074656
14685076501496281923240, 10015350798743319747707033007600, 12378523459120956991548018324, 1452229009170615066047532876, 1613587787967350073380
614764, 16938214348828536681954036, 1675208012521503627885564, 155620416463746238656036, 13532210127282281622264, 1097206226536401212616,
82585414900589338584, 5741232051912627816, 366461620334848584, 21328454093562616, 1122550215450664, 52895036330136, 2203959847089, 7993636
7511, 2472258789, 63391251, 1293699, 19701, 199, 1]
```

- ## PS1_4



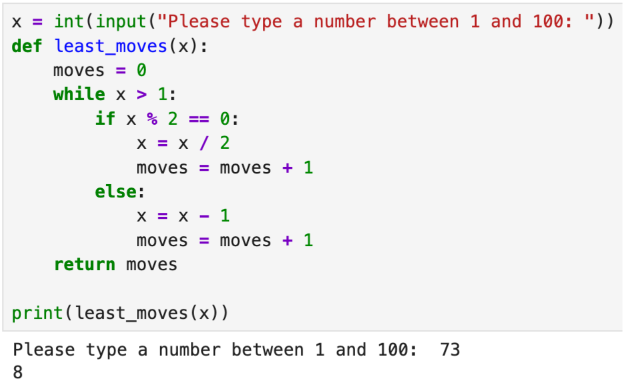
```
x = int(input("Please type a number between 1 and 100: "))
def least_moves(x):
    moves = 0
    while x > 1:
        if x % 2 == 0:
            x = x / 2
            moves = moves + 1
        else:
            x = x - 1
            moves = moves + 1
    return moves

print(least_moves(x))
```

```
Please type a number between 1 and 100:  2
1
```



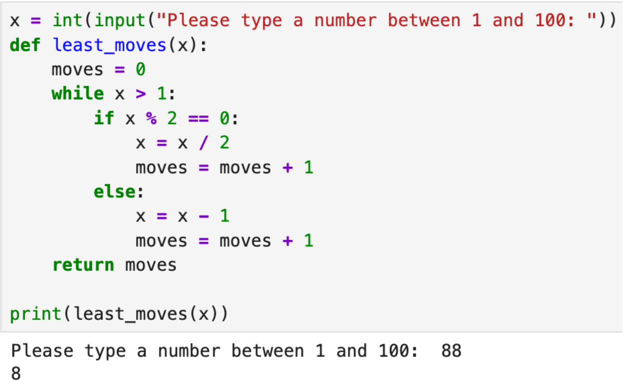
```
x = int(input("Please type a number between 1 and 100: "))
def least_moves(x):
    moves = 0
    while x > 1:
        if x % 2 == 0:
            x = x / 2
            moves = moves + 1
        else:
            x = x - 1
            moves = moves + 1
    return moves

print(least_moves(x))
```

```
Please type a number between 1 and 100:  5
3
```



```
x = int(input("Please type a number between 1 and 100: "))
def least_moves(x):
    moves = 0
    while x > 1:
        if x % 2 == 0:
            x = x / 2
            moves = moves + 1
        else:
            x = x - 1
            moves = moves + 1
    return moves

print(least_moves(x))
```

```
Please type a number between 1 and 100:  73
8
```



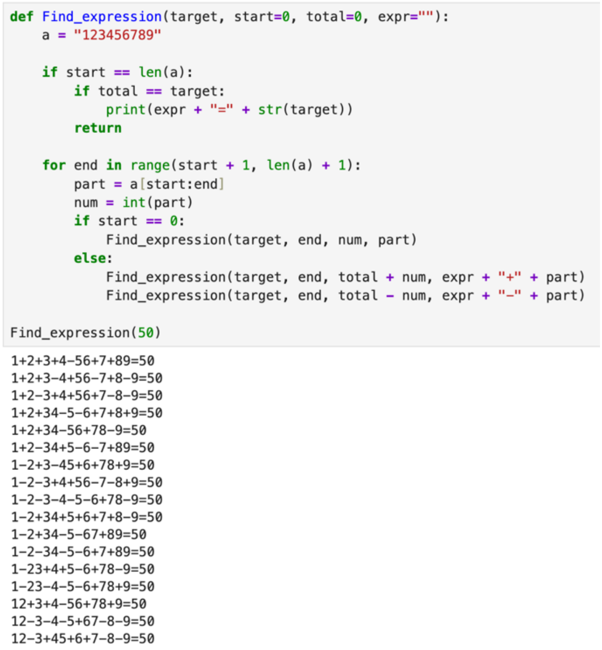
```
x = int(input("Please type a number between 1 and 100: "))
def least_moves(x):
    moves = 0
    while x > 1:
        if x % 2 == 0:
            x = x / 2
            moves = moves + 1
        else:
            x = x - 1
            moves = moves + 1
    return moves

print(least_moves(x))
```

```
Please type a number between 1 and 100:  88
8
```

The first 2 outputs are to verify the result given in the assignment, detailed notes about the code could be found near the scripts.

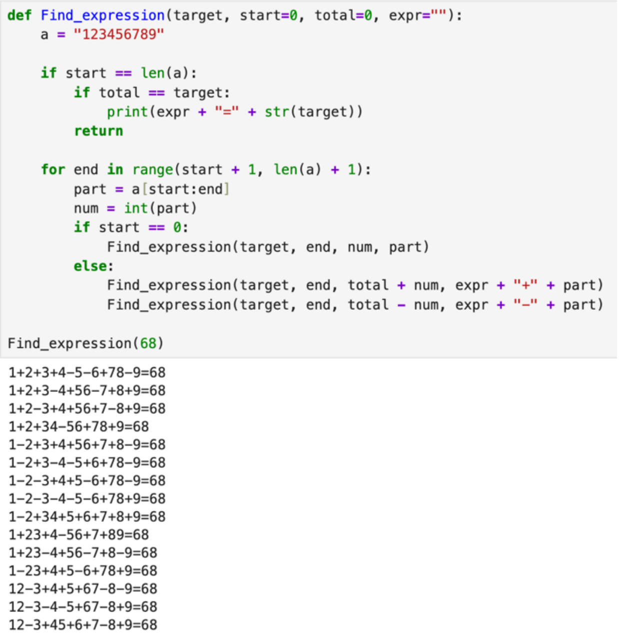
- ## PS1_5

  I got explained about how to understand and break these questions in to parts by my friend Haochen, Yuan, who is also a PhD from PolyU.



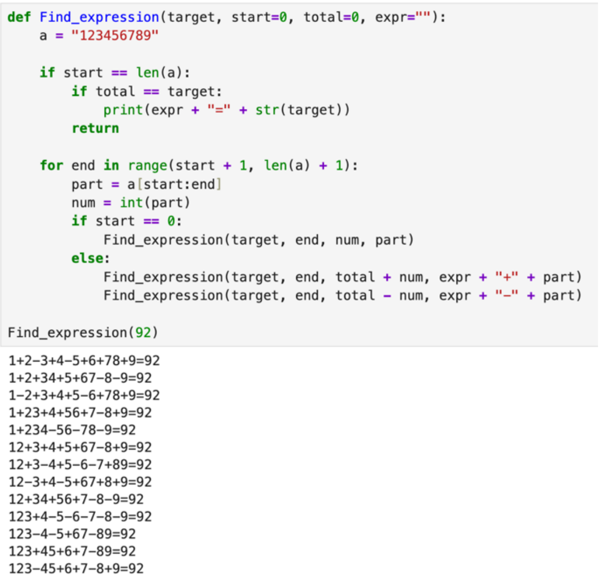
```
def Find_expression(target, start=0, total=0, expr=""):
    a = "123456789"

    if start == len(a):
        if total == target:
            print(expr + "=" + str(target))
        return

    for end in range(start + 1, len(a) + 1):
        part = a[start:end]
        num = int(part)
        if start == 0:
            Find_expression(target, end, num, part)
        else:
            Find_expression(target, end, total + num, expr + "+" + part)
            Find_expression(target, end, total - num, expr + "-" + part)

Find_expression(50)
```

```
1+2+3+4-56+7+89=50
1+2+3-4+56-7+8-9=50
1+2-3+4+56+7-8-9=50
1+2+34-5-6+7+8+9=50
1+2+34-56+78-9=50
1+2-34+5-6-7+89=50
1-2+3-45+6+78+9=50
1-2-3+4+56-7-8+9=50
1-2-3-4-5-6+78-9=50
1-2+34+5+6+7+8-9=50
1-2+34-5-67+89=50
1-2-34-5-6+7+89=50
1-23+4+5-6+78-9=50
1-23-4-5-6+78+9=50
12+3+4-56+78+9=50
12-3-4-5+67-8-9=50
12-3+45+6+7-8-9=50
```



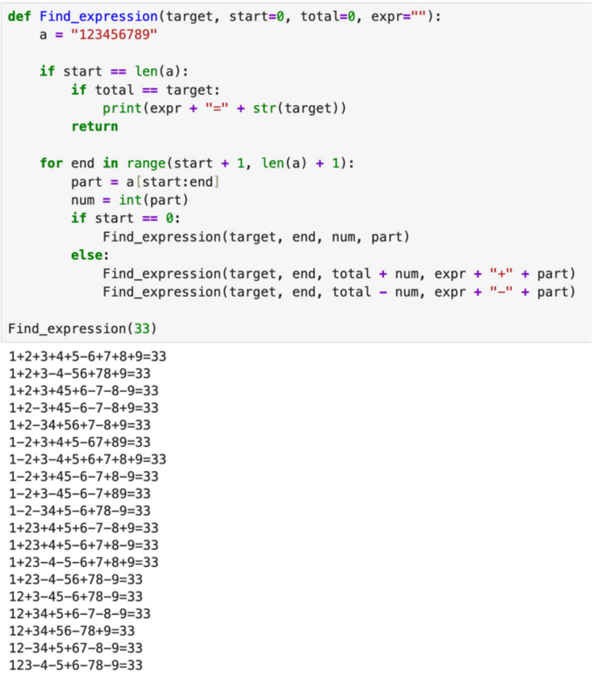
```
def Find_expression(target, start=0, total=0, expr=""):
    a = "123456789"

    if start == len(a):
        if total == target:
            print(expr + "=" + str(target))
        return

    for end in range(start + 1, len(a) + 1):
        part = a[start:end]
        num = int(part)
        if start == 0:
            Find_expression(target, end, num, part)
        else:
            Find_expression(target, end, total + num, expr + "+" + part)
            Find_expression(target, end, total - num, expr + "-" + part)

Find_expression(68)
```

```
1+2+3+4-5-6+78-9=68
1+2+3-4+56-7+8+9=68
1+2-3+4+56+7-8+9=68
1+2+34-56+78+9=68
1-2+3+4+56+7+8-9=68
1-2+3-4-5+6+78-9=68
1-2-3+4+5-6+78-9=68
1-2-3-4-5-6+78+9=68
1-2+34+5+6+7+8+9=68
1+23+4-56+7+89=68
1+23-4+56-7+8-9=68
1-23+4+5-6+78+9=68
12-3+4+5+67-8-9=68
12-3-4-5+67-8+9=68
12-3+45+6+7-8+9=68
```



```python
def Find_expression(target, start=0, total=0, expr=""):
    a = "123456789"

    if start == len(a):
        if total == target:
            print(expr + "=" + str(target))
        return

    for end in range(start + 1, len(a) + 1):
        part = a[start:end]
        num = int(part)
        if start == 0:
            Find_expression(target, end, num, part)
        else:
            Find_expression(target, end, total + num, expr + "+" + part)
            Find_expression(target, end, total - num, expr + "-" + part)


Find_expression(92)
```

```
1+2-3+4-5+6+78+9=92
1+2+34+5+67-8-9=92
1-2+3+4+5-6+78+9=92
1+23+4+56+7-8+9=92
1+234-56-78-9=92
12+3+4+5+67-8+9=92
12+3-4+5-6-7+89=92
12-3+4-5+67+8+9=92
12+34+56+7-8-9=92
123+4-5-6-7-8-9=92
123-4-5+67-89=92
123+45+6+7-89=92
123-45+6+7-8+9=92
```



```python
def Find_expression(target, start=0, total=0, expr=""):
    a = "123456789"

    if start == len(a):
        if total == target:
            print(expr + "=" + str(target))
        return

    for end in range(start + 1, len(a) + 1):
        part = a[start:end]
        num = int(part)
        if start == 0:
            Find_expression(target, end, num, part)
        else:
            Find_expression(target, end, total + num, expr + "+" + part)
            Find_expression(target, end, total - num, expr + "-" + part)


Find_expression(33)
```

```
1+2+3+4+5-6+7+8+9=33
1+2+3-4-56+78+9=33
1+2+3+45+6-7-8-9=33
1+2-3+45-6-7-8+9=33
1+2-34+56+7-8+9=33
1-2+3+4+5-67+89=33
1-2+3-4+5+6+7+8+9=33
1-2+3+45-6-7+8-9=33
1-2+3-45-6-7+89=33
1-2-34+5-6+78-9=33
1+23+4+5+6-7-8+9=33
1+23+4+5-6+7+8-9=33
1+23-4-5-6+7+8+9=33
1+23-4-56+78-9=33
12+3-45-6+78-9=33
12+34+5+6-7-8-9=33
12+34+56-78+9=33
12-34+5+67-8-9=33
123-4-5+6-78-9=33
```

Above are the results generated by the script, I choose four random numbers from 1 to 100, the first “if” condition is to end the loop, the second for loop is to index the numbers in the string and the if condition is to call the function internally, start ==0 index the first number “1”.